# TIỂU LUẬN SoAD

Kiến trúc và Thiết kế Phần mềm

Tài liệu này dành cho sinh viên học môn SoAD tham khảo

Xây dựng Hệ thống E-Commerce theo Microservices và AI

GVHD: Trần Đình Quế

Ngày 15 tháng 4 năm 2026

# Mục lục


**1 Từ Monolithic đến Microservices và DDD** **6**
1.1 Giới thiệu Monolithic Architecture . . . 6
1.1.1 Khái niệm . . . 6
1.1.2 Cấu trúc điển hình . . . 6
1.1.3 Ví dụ thực tế . . . 6
1.1.4 Nhược điểm chi tiết . . . 6
1.1.5 Khi nào nên dùng Monolithic . . . 7
1.2 Microservices Architecture . . . 7
1.2.1 Khái niệm . . . 7
1.2.2 Đặc điểm . . . 7
1.2.3 So sánh Monolithic vs Microservices . . . 7
1.2.4 Ưu điểm . . . 7
1.2.5 Nhược điểm . . . 7
1.2.6 Nguyên tắc thiết kế . . . 7
1.3 Domain Driven Design (DDD) . . . 8
1.3.1 Mục tiêu . . . 8
1.3.2 Các khái niệm cốt lõi . . . 8
1.3.3 Context Map . . . 8
1.3.4 DDD trong Microservices . . . 8
1.4 Case Study: Healthcare (Luyện Decomposition) . . . 9
1.4.1 Mô tả bài toán . . . 9
1.4.2 Bước 1: Xác định Domain . . . 9
1.4.3 Bước 2: Xác định Bounded Context . . . 9
1.4.4 Bước 3: Phân rã thành Microservices . . . 9
1.4.5 Bước 4: Xác định quan hệ . . . 9
1.4.6 Ví dụ API . . . 9
1.5 Kết luận . . . 10

**2 Phát triển Hệ E-Commerce Microservices** **11**
2.1 Xác định yêu cầu . . . 11
2.1.1 Functional Requirements . . . 11
2.1.2 Non-functional Requirements . . . 11
2.2 Phân rã hệ thống theo DDD . . . 11
2.2.1 Bounded Context . . . 11
2.2.2 Nguyên tắc . . . 12
2.3 Thiết kế Product Service (Django) . . . 12
2.3.1 Phân loại sản phẩm . . . 12
2.3.2 Model tổng quát . . . 12

2.3.3 Chi tiết theo domain . . . 12
2.3.4 API . . . 13
2.4 Thiết kế User Service (Django) . . . 13
2.4.1 Phân loại người dùng . . . 13
2.4.2 Model . . . 13
2.4.3 Phân quyền (RBAC) . . . 13
2.4.4 API . . . 13
2.5 Thiết kế Cart Service . . . 14
2.5.1 Model . . . 14
2.5.2 Logic . . . 14
2.5.3 API . . . 14
2.6 Thiết kế Order Service . . . 14
2.6.1 Model . . . 14
2.6.2 Workflow . . . 14
2.7 Thiết kế Payment Service . . . 15
2.7.1 Model . . . 15
2.7.2 Trạng thái . . . 15
2.7.3 API . . . 15
2.8 Thiết kế Shipping Service . . . 15
2.8.1 Model . . . 15
2.8.2 Trạng thái . . . 15
2.8.3 API . . . 15
2.9 Luồng hệ thống tổng thể . . . 16
2.10 Hướng dẫn thực hành . . . 16
2.10.1 Mục tiêu . . . 16
2.10.2 Hướng dẫn vẽ Class Diagram bằng Visual Paradigm . . . 16
2.10.3 Mapping Class Diagram sang Database . . . 17
2.10.4 Thiết kế Database cho từng Service . . . 17
2.10.5 So sánh MySQL vs PostgreSQL . . . 19
2.10.6 Bài tập . . . 20
2.10.7 Checklist đánh giá . . . 20
2.11 Kết luận . . . 20

**3 AI Service cho tư vấn sản phẩm 21**
3.1 Mục tiêu . . . 21
3.2 Kiến trúc AI Service . . . 21
3.3 Thu thập dữ liệu . . . 21
3.3.1 User Behavior Data . . . 21
3.3.2 Ví dụ dataset . . . 22
3.4 Mô hình LSTM (Sequence Modeling) . . . 22
3.4.1 Ý tưởng . . . 22
3.4.2 Model chi tiết . . . 22
3.4.3 Training . . . 22
3.5 Knowledge Graph với Neo4j . . . 22
3.5.1 Mô hình đồ thị . . . 22
3.5.2 Ví dụ Cypher . . . 23
3.5.3 Truy vấn gợi ý . . . 23
3.6 RAG (Retrieval-Augmented Generation) . . . 23

3.6.1 Pipeline . . . 23
3.6.2 Vector Database . . . 23
3.6.3 Ví dụ . . . 23
3.7 Kết hợp Hybrid Model . . . 23
3.8 Hai dạng AI Service . . . 24
3.8.1 1. Recommendation List . . . 24
3.8.2 2. Chatbot tư vấn . . . 24
3.9 Triển khai AI Service . . . 24
3.9.1 Tech stack . . . 24
3.9.2 Kiến trúc . . . 25
3.10 Bài tập . . . 25
3.11 Checklist đánh giá . . . 25
3.12 Kết luận . . . 25

**4 Xây dựng hệ thống hoàn chỉnh 26**
4.1 Kiến trúc tổng thể . . . 26
4.1.1 Mô hình hệ thống . . . 26
4.1.2 Nguyên tắc . . . 26
4.2 System Architecture . . . 26
4.2.1 Overview . . . 26
4.2.2 Microservice Architecture . . . 27
4.2.3 API Gateway . . . 27
4.2.4 Service Communication . . . 27
4.2.5 Containerization and Deployment . . . 27
4.2.6 System Structure . . . 28
4.2.7 Design Principles . . . 28
4.2.8 Security Considerations . . . 28
4.2.9 Discussion . . . 28
4.3 API Gateway (Nginx) . . . 29
4.3.1 Vai trò . . . 29
4.3.2 Cấu hình mẫu . . . 29
4.4 Authentication (JWT) . . . 29
4.4.1 Cài đặt . . . 29
4.4.2 Cấu hình . . . 29
4.4.3 Luồng . . . 29
4.5 Giao tiếp giữa các Service . . . 29
4.5.1 REST API call . . . 29
4.5.2 Best Practice . . . 30
4.6 Docker hóa hệ thống . . . 30
4.6.1 Dockerfile (Django) . . . 30
4.6.2 docker-compose.yml . . . 30
4.7 Luồng hệ thống (End-to-End) . . . 30
4.7.1 Use case: Mua hàng . . . 30
4.7.2 Sequence logic . . . 31
4.8 Triển khai Kubernetes (Optional) . . . 31
4.8.1 Deployment . . . 31
4.8.2 Service . . . 31
4.9 Logging và Monitoring . . . 31

4.10 Đánh giá hệ thống . . . . . 31
4.10.1 Hiệu năng . . . . . 31
4.10.2 Khả năng mở rộng . . . . . 31
4.10.3 Ưu điểm . . . . . 31
4.10.4 Nhược điểm . . . . . 32
4.11 Bài tập thực hành . . . . . 32
4.12 Checklist đánh giá . . . . . 32

# Hạn nộp các version

- NỘP VERSION 01 TRƯỚC 23:30, THỨ HAI 27/04
  tieuluan.v01_Lớp.nhóm_tenHosinhvien.PDF
- NỘP VERSION 02 TRƯỚC 23:30, THỨ HAI 11/05
  tieuluan.v02_Lớp.nhóm_tenHosinhvien.PDF
- NỘP VERSION FINAL TRƯỚC 23:30, TRƯỚC NGÀY THI 1 NGÀY
  tieuluan.final_Lớp.nhóm_tenHosinhvien.PDF
- SINH VIÊN MANG THEO BẢN IN KHI ĐI THI
  **Bản IN được viết tay hay vẽ tay nhiều là điểm +**

# Chương 1

# Từ Monolithic đến Microservices và DDD

## 1.1 Giới thiệu Monolithic Architecture

### 1.1.1 Khái niệm

Monolithic Architecture là mô hình trong đó toàn bộ hệ thống (UI, business logic, database access) được xây dựng và triển khai như một khối duy nhất.

### 1.1.2 Cấu trúc điển hình

- Presentation Layer (UI)
- Business Logic Layer
- Data Access Layer

### 1.1.3 Ví dụ thực tế

Một hệ e-commerce monolithic sẽ bao gồm:

- Quản lý sản phẩm
- Giỏ hàng
- Thanh toán
- User

tất cả nằm trong cùng một codebase.

### 1.1.4 Nhược điểm chi tiết

- **Khó mở rộng:** scale toàn hệ thống thay vì từng module
- **Coupling cao:** thay đổi nhỏ ảnh hưởng toàn bộ hệ thống
- **Deploy rủi ro:** lỗi nhỏ có thể làm sập toàn hệ
- **Khó phát triển nhóm:** nhiều team cùng sửa một codebase

### 1.1.5 Khi nào nên dùng Monolithic

- Hệ thống nhỏ
- MVP (Minimum Viable Product)
- Team ít người

## 1.2 Microservices Architecture

### 1.2.1 Khái niệm

Microservices là kiến trúc chia hệ thống thành các dịch vụ nhỏ, độc lập, mỗi service thực hiện một chức năng riêng biệt.

### 1.2.2 Đặc điểm

- Mỗi service có database riêng
- Giao tiếp qua API (REST/gRPC)
- Deploy độc lập

### 1.2.3 So sánh Monolithic vs Microservices

| Tiêu chí | Monolithic | Microservices |
|---|---|---|
| Deploy | Một lần | Nhiều lần |
| Scale | Toàn hệ | Từng service |
| Coupling | Cao | Thấp |

### 1.2.4 Ưu điểm

- Scale độc lập từng service
- Tăng tốc phát triển
- Phù hợp hệ thống lớn

### 1.2.5 Nhược điểm

- Phức tạp hệ thống
- Quản lý distributed system
- Debug khó hơn

### 1.2.6 Nguyên tắc thiết kế

- Single Responsibility
- Loose Coupling
- High Cohesion

## 1.3 Domain Driven Design (DDD)

### 1.3.1 Mục tiêu

DDD giúp mô hình hóa hệ thống theo nghiệp vụ (business domain), không phụ thuộc công nghệ.

### 1.3.2 Các khái niệm cốt lõi

**Entity**

Đối tượng có định danh (ID).

- Ví dụ: User, Product

**Value Object**

Không có ID, so sánh bằng giá trị.

- Ví dụ: Address, Money

**Aggregate**

Nhóm các entity liên quan.

- Ví dụ: Order + OrderItem

**Bounded Context**

Ranh giới domain rõ ràng.

- Ví dụ: User Context, Order Context

### 1.3.3 Context Map

Sinh viên cần vẽ sơ đồ thể hiện mối quan hệ giữa các bounded context:

- Shared Kernel
- Customer-Supplier

### 1.3.4 DDD trong Microservices

- Mỗi bounded context = 1 microservice
- Tránh chia service theo technical layer

## 1.4 Case Study: Healthcare (Luyện Decomposition)

### 1.4.1 Mô tả bài toán

Hệ thống quản lý bệnh viện gồm:

- Quản lý bệnh nhân
- Quản lý bác sĩ
- Đặt lịch khám

### 1.4.2 Bước 1: Xác định Domain

- Patient Management
- Doctor Management
- Appointment Scheduling

### 1.4.3 Bước 2: Xác định Bounded Context

- Patient Context
- Doctor Context
- Appointment Context

### 1.4.4 Bước 3: Phân rã thành Microservices

- Patient Service
- Doctor Service
- Appointment Service

### 1.4.5 Bước 4: Xác định quan hệ

- Appointment phụ thuộc Patient và Doctor
- Giao tiếp qua API

### 1.4.6 Ví dụ API

```
GET /patients
GET /doctors
POST /appointments
```

## 1.5 Kết luận

- Monolithic phù hợp hệ nhỏ
- Microservices phù hợp hệ lớn
- DDD là nền tảng quan trọng để phân rã hệ thống đúng

# Chương 2

# Phát triển Hệ E-Commerce Microservices

## 2.1 Xác định yêu cầu

### 2.1.1 Functional Requirements

- Quản lý sản phẩm (đa domain: book, electronics, fashion)
- Quản lý người dùng (admin, staff, customer)
- Giỏ hàng (cart)
- Đặt hàng (order)
- Thanh toán (payment)
- Giao hàng (shipping)
- Tìm kiếm và gợi ý sản phẩm

### 2.1.2 Non-functional Requirements

- Scalability: scale từng service độc lập
- High Availability: hệ thống luôn sẵn sàng
- Security: JWT, authentication
- Maintainability: dễ bảo trì

## 2.2 Phân rã hệ thống theo DDD

### 2.2.1 Bounded Context

- User Context → user-service
- Product Context → product-service
- Cart Context → cart-service

- Order Context → order-service
- Payment Context → payment-service
- Shipping Context → shipping-service

### 2.2.2 Nguyên tắc

- Mỗi context = 1 database riêng
- Giao tiếp qua REST API

## 2.3 Thiết kế Product Service (Django)

### 2.3.1 Phân loại sản phẩm

- **Book**: giáo trình, tiểu thuyết
- **Electronics**: mobile, laptop, tủ lạnh, điều hòa
- **Fashion**: áo, quần, giày

### 2.3.2 Model tổng quát

```
class Category(models.Model):
    name = models.CharField(max_length=100)

class Product(models.Model):
    name = models.CharField(max_length=255)
    price = models.FloatField()
    stock = models.IntegerField()
    category = models.ForeignKey(Category, on_delete=models.
       CASCADE)
```

### 2.3.3 Chi tiết theo domain

**Book**

```
class Book(models.Model):
    product = models.OneToOneField(Product, on_delete=models.
       CASCADE)
    author = models.CharField(max_length=255)
    publisher = models.CharField(max_length=255)
    isbn = models.CharField(max_length=20)
```

**Electronics**

```
class Electronics(models.Model):
    product = models.OneToOneField(Product, on_delete=models.
       CASCADE)
    brand = models.CharField(max_length=100)
    warranty = models.IntegerField()
```

**Fashion**

```
class Fashion(models.Model):
    product = models.OneToOneField(Product, on_delete=models.
       CASCADE)
    size = models.CharField(max_length=10)
    color = models.CharField(max_length=50)
```

### 2.3.4 API

```
GET /products/
POST /products/
GET /products/{id}
```

## 2.4 Thiết kế User Service (Django)

### 2.4.1 Phân loại người dùng

- Admin: toàn quyền hệ thống
- Staff: xử lý đơn hàng, vận hành
- Customer: mua hàng

### 2.4.2 Model

```
from django.contrib.auth.models import AbstractUser

class User(AbstractUser):
    ROLE_CHOICES = (
        ('admin', 'Admin'),
        ('staff', 'Staff'),
        ('customer', 'Customer'),
    )
    role = models.CharField(max_length=20, choices=ROLE_CHOICES)
```

### 2.4.3 Phân quyền (RBAC)

- Admin: CRUD toàn bộ
- Staff: xử lý order, shipping
- Customer: mua hàng, xem sản phẩm

### 2.4.4 API

```
POST /auth/register
POST /auth/login
GET /users/
```

## 2.5 Thiết kế Cart Service

### 2.5.1 Model

```
class Cart(models.Model):
    user_id = models.IntegerField()

class CartItem(models.Model):
    cart = models.ForeignKey(Cart, on_delete=models.CASCADE)
    product_id = models.IntegerField()
    quantity = models.IntegerField()
```

### 2.5.2 Logic

- Add product vào cart
- Update số lượng
- Remove item

### 2.5.3 API

```
POST /cart/add
GET /cart/
DELETE /cart/remove
```

## 2.6 Thiết kế Order Service

### 2.6.1 Model

```
class Order(models.Model):
    user_id = models.IntegerField()
    total_price = models.FloatField()
    status = models.CharField(max_length=50)

class OrderItem(models.Model):
    order = models.ForeignKey(Order, on_delete=models.CASCADE)
    product_id = models.IntegerField()
    quantity = models.IntegerField()
```

### 2.6.2 Workflow

- Tạo order từ cart
- Gửi request sang payment-service
- Sau khi thanh toán → shipping

## 2.7 Thiết kế Payment Service

### 2.7.1 Model

```
class Payment(models.Model):
    order_id = models.IntegerField()
    amount = models.FloatField()
    status = models.CharField(max_length=50)
```

### 2.7.2 Trạng thái

- Pending
- Success
- Failed

### 2.7.3 API

```
POST /payment/pay
GET /payment/status
```

## 2.8 Thiết kế Shipping Service

### 2.8.1 Model

```
class Shipment(models.Model):
    order_id = models.IntegerField()
    address = models.TextField()
    status = models.CharField(max_length=50)
```

### 2.8.2 Trạng thái

- Processing
- Shipping
- Delivered

### 2.8.3 API

```
POST /shipping/create
GET /shipping/status
```

## 2.9 Luồng hệ thống tổng thể

1. User đăng nhập (user-service)
2. Xem sản phẩm (product-service)
3. Thêm vào giỏ hàng (cart-service)
4. Tạo đơn hàng (order-service)
5. Thanh toán (payment-service)
6. Giao hàng (shipping-service)

## 2.10 Hướng dẫn thực hành

### 2.10.1 Mục tiêu

Sinh viên cần:

- Thiết kế Class Diagram bằng Visual Paradigm (VP)
- Xây dựng database cho từng microservice
- Mapping từ Class Diagram → Database

### 2.10.2 Hướng dẫn vẽ Class Diagram bằng Visual Paradigm

**Bước 1: Xác định lớp (Classes)**

Sinh viên xác định các lớp chính theo từng service:

**Product Service:**

- Product
- Category
- Book
- Electronics
- Fashion

**User Service:**

- User
- Role

**Order Service:**

- Order
- OrderItem

**Bước 2: Xác định thuộc tính**

Ví dụ lớp Product:

- id: int
- name: string
- price: float
- stock: int

**Bước 3: Xác định quan hệ (Relationships)**

- **Association:** Product → Category
- **Inheritance:** Book, Electronics, Fashion kế thừa Product
- **Composition:** Order chứa OrderItem

**Ký hiệu UML**

- 1..* (one-to-many)
- 1..1 (one-to-one)

**Yêu cầu bài nộp**

- Export sơ đồ từ VP (PNG/PDF)
- Có đầy đủ class + relationship

### 2.10.3 Mapping Class Diagram sang Database

**Nguyên tắc**

- Class → Table
- Attribute → Column
- Relationship → Foreign Key

**Ví dụ**

```
Product(id, name, price)
Category(id, name)

=> Product.category_id (FK)
```

### 2.10.4 Thiết kế Database cho từng Service

**Nguyên tắc Microservices**

- Mỗi service có database riêng với data model tương ứng với Biểu đồ lớp
- Không share database giữa các service

**1. Product Service Database (PostgreSQL)**

**Lý do chọn PostgreSQL:**

- Hỗ trợ tốt JSON
- Phù hợp dữ liệu phức tạp

```
CREATE TABLE category (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100)
);

CREATE TABLE product (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255),
    price FLOAT,
    stock INT,
    category_id INT REFERENCES category(id)
);

CREATE TABLE book (
    product_id INT PRIMARY KEY,
    author VARCHAR(255),
    isbn VARCHAR(20)
);
```

**2. User Service Database (MySQL)**

**Lý do chọn MySQL:**

- Phổ biến
- Phù hợp authentication

```
CREATE TABLE user (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(100),
    password VARCHAR(255),
    role VARCHAR(20)
);
```

**3. Cart Service**

```
CREATE TABLE cart (
    id INT PRIMARY KEY,
    user_id INT
);

CREATE TABLE cart_item (
    id INT PRIMARY KEY,
```

```
    cart_id INT,
    product_id INT,
    quantity INT
);
```

**4. Order Service**

```
CREATE TABLE orders (
    id INT PRIMARY KEY,
    user_id INT,
    total_price FLOAT,
    status VARCHAR(50)
);

CREATE TABLE order_item (
    id INT PRIMARY KEY,
    order_id INT,
    product_id INT,
    quantity INT
);
```

**5. Payment Service**

```
CREATE TABLE payment (
    id INT PRIMARY KEY,
    order_id INT,
    amount FLOAT,
    status VARCHAR(50)
);
```

**6. Shipping Service**

```
CREATE TABLE shipment (
    id INT PRIMARY KEY,
    order_id INT,
    address TEXT,
    status VARCHAR(50)
);
```

### 2.10.5 So sánh MySQL vs PostgreSQL

| Tiêu chí | MySQL | PostgreSQL |
|---|---|---|
| Hiệu năng | Tốt | Tốt |
| JSON | Trung bình | Mạnh |
| Quan hệ phức tạp | Trung bình | Tốt |

### 2.10.6 Bài tập

- Vẽ Class Diagram cho toàn bộ hệ thống bằng VP
- Mapping sang database schema
- Triển khai database bằng MySQL/PostgreSQL

### 2.10.7 Checklist đánh giá

- Có sơ đồ class đúng UML
- Có mapping rõ ràng sang database
- Database tách riêng từng service
- Có sử dụng cả MySQL và PostgreSQL

## 2.11 Kết luận

- Kiến trúc microservices giúp hệ thống linh hoạt
- Django phù hợp xây dựng nhanh
- DDD giúp thiết kế đúng ngay từ đầu

# Chương 3

# AI Service cho tư vấn sản phẩm

## 3.1 Mục tiêu

Xây dựng hệ thống AI gợi ý sản phẩm dựa trên:

- Hành vi người dùng (click, search, add-to-cart)
- Quan hệ sản phẩm (similarity)
- Ngữ cảnh truy vấn (chatbot)

**Output:**

- Danh sách sản phẩm đề xuất
- Chatbot tư vấn

## 3.2 Kiến trúc AI Service

AI Service được thiết kế như một microservice độc lập:

- Input: user behavior, query
- Processing:
  - LSTM model
  - Knowledge Graph
  - RAG
- Output: recommendation / chatbot response

## 3.3 Thu thập dữ liệu

### 3.3.1 User Behavior Data

- user_id
- product_id
- action (view, click, add_to_cart)
- timestamp

### 3.3.2 Ví dụ dataset

```
user_id, product_id, action, time
1, 101, view, t1
1, 102, add_to_cart, t2
```

## 3.4 Mô hình LSTM (Sequence Modeling)

### 3.4.1 Ý tưởng

Dự đoán sản phẩm tiếp theo dựa trên chuỗi hành vi.

### 3.4.2 Model chi tiết

```
import torch
import torch.nn as nn

class LSTMModel(nn.Module):
    def __init__(self, input_dim=10, hidden_dim=64, output_dim
       =100):
        super().__init__()
        self.lstm = nn.LSTM(input_dim, hidden_dim, batch_first=
           True)
        self.fc = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        out, _ = self.lstm(x)
        out = out[:, -1, :]
        return self.fc(out)
```

### 3.4.3 Training

```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters())

for epoch in range(epochs):
    output = model(x)
    loss = criterion(output, y)
    loss.backward()
    optimizer.step()
```

## 3.5 Knowledge Graph với Neo4j

### 3.5.1 Mô hình đồ thị

- Node: User, Product
- Edge:

  - BUY
  - VIEW
  - SIMILAR

### 3.5.2 Ví dụ Cypher

```
CREATE (u:User {id:1})
CREATE (p:Product {id:101})
CREATE (u)-[:BUY]->(p)
```

### 3.5.3 Truy vấn gợi ý

```
MATCH (u:User {id:1})-[:BUY]->(p)-[:SIMILAR]->(rec)
RETURN rec
```

## 3.6 RAG (Retrieval-Augmented Generation)

### 3.6.1 Pipeline

- Retrieve:
  - Tìm sản phẩm liên quan từ DB / vector DB
- Generate:
  - Sinh câu trả lời bằng LLM

### 3.6.2 Vector Database

- FAISS / ChromaDB
- Embedding từ mô tả sản phẩm

### 3.6.3 Ví dụ

```
query = "laptop gaming"
results = vector_db.search(query)
response = LLM.generate(results)
```

## 3.7 Kết hợp Hybrid Model

- LSTM: dự đoán hành vi
- Graph: quan hệ sản phẩm
- RAG: hiểu ngữ nghĩa

**Final Recommendation:**

```
final_score = w1 * lstm + w2 * graph + w3 * rag
```

## 3.8 Hai dạng AI Service

### 3.8.1 1. Recommendation List

**Use cases**

- Khi search
- Khi add-to-cart

**API**

```
GET /recommend?user_id=1
```

**Output**

```
[101, 102, 205]
```

### 3.8.2 2. Chatbot tư vấn

**Input**

"tôi cần laptop giá rẻ"

**Pipeline**

- NLP hiểu intent
- Retrieve sản phẩm
- Generate response

**API**

```
POST /chatbot
```

**Output**

"Bạn có thể tham khảo Laptop XYZ giá 10 triệu..."

## 3.9 Triển khai AI Service

### 3.9.1 Tech stack

- tensorflow/PyTorch (LSTM)
- Neo4j (Graph)
- FAISS (Vector DB)
- FastAPI (service)

### 3.9.2 Kiến trúc

- AI service độc lập
- Giao tiếp với các service khác qua API

## 3.10 Bài tập

- Xây dựng model LSTM đơn giản
- Tạo graph trong Neo4j
- Implement API recommendation
- Xây dựng chatbot cơ bản

## 3.11 Checklist đánh giá

- Có pipeline AI rõ ràng
- Có model (LSTM)
- Có Graph và RAG
- Có API hoạt động

## 3.12 Kết luận

- AI giúp cá nhân hóa trải nghiệm
- Kết hợp nhiều mô hình cho hiệu quả cao
- Phù hợp hệ e-commerce hiện đại

# Chương 4

# Xây dựng hệ thống hoàn chỉnh

## 4.1 Kiến trúc tổng thể

### 4.1.1 Mô hình hệ thống

Hệ thống được xây dựng theo kiến trúc microservices, mỗi service là một Django project độc lập.

- API Gateway (Nginx)
- user-service (Django)
- product-service (Django)
- cart-service (Django)
- order-service (Django)
- payment-service (Django)
- shipping-service (Django)
- ai-service (FastAPI/Python)

### 4.1.2 Nguyên tắc

- Mỗi service có database riêng
- Giao tiếp qua REST API
- Không truy cập DB của service khác

## 4.2 System Architecture

### 4.2.1 Overview

The proposed system, named `ecom-final`, is designed as a fully distributed microservice-based e-commerce platform. The architecture follows modern enterprise design principles, ensuring scalability, maintainability, and fault isolation.

Each core business domain is implemented as an independent Django REST microservice, while an API Gateway is employed to manage request routing, authentication, and system-wide policies.

### 4.2.2 Microservice Architecture

The system consists of the following core services:

- **User Service**: Handles authentication, authorization, and user management.
- **Product Service**: Manages product catalog, categories, and inventory.
- **Order Service**: Processes customer orders and order lifecycle.
- **Payment Service**: Handles payment transactions and billing.
- **Notification Service**: Sends asynchronous notifications (email, SMS).

Each service is independently deployable and maintains its own database, following the principle of database-per-service.

### 4.2.3 API Gateway

An API Gateway layer is introduced as the single entry point for all client requests. The gateway is responsible for:

- Routing incoming requests to appropriate microservices
- Handling authentication using JSON Web Tokens (JWT)
- Enforcing rate limiting and security policies
- Logging and monitoring API usage

In this system, the API Gateway is implemented using NGINX as a reverse proxy.

### 4.2.4 Service Communication

The system adopts a hybrid communication strategy:

- **Synchronous communication**: RESTful APIs over HTTP for real-time operations (e.g., order validation).
- **Asynchronous communication**: Message queues (e.g., Redis, RabbitMQ) for event-driven workflows.

For example, when an order is created, an event is published and consumed by the payment and notification services.

### 4.2.5 Containerization and Deployment

All services are containerized using Docker to ensure consistency across environments. The system is orchestrated using Docker Compose for development and can be extended to Kubernetes for production deployment.

### 4.2.6 System Structure

```
ecom-final/

 |-- gateway/
 |   |-- nginx.conf                      //THỂ HIỆN QUAN TRỌNG

 |-- user-service/                       //staff, admin, customer
 |-- product-service/                  //10 nhóm loại sản phẩm
 |-- cart-service/
 |-- order-service/
 |-- payment-service/
 |-- ai-service/

 |-- infrastructure/
 |   |-- docker-compose.yml
```

Hình 4.1: Microservice architecture of the `ecom-final` system

### 4.2.7 Design Principles

The proposed architecture adheres to the following principles:

- **Loose Coupling**: Services interact only through APIs or messaging systems.
- **High Cohesion**: Each service encapsulates a single business domain.
- **Scalability**: Services can be scaled independently.
- **Fault Isolation**: Failure in one service does not affect others.

### 4.2.8 Security Considerations

Security is enforced through:

- JWT-based authentication
- API Gateway validation
- Role-based access control (RBAC)

### 4.2.9 Discussion

Compared to monolithic architectures, the proposed microservice design significantly improves system flexibility and scalability. However, it introduces additional complexity in deployment and service coordination, which is mitigated through containerization and standardized communication protocols.

## 4.3 API Gateway (Nginx)

### 4.3.1 Vai trò

- Entry point cho toàn hệ thống
- Routing request đến đúng service
- Xử lý authentication

### 4.3.2 Cấu hình mẫu

```
location /users/ {
    proxy_pass http://user-service:8000;
}

location /products/ {
    proxy_pass http://product-service:8001;
}
```

## 4.4 Authentication (JWT)

### 4.4.1 Cài đặt

```
pip install djangorestframework-simplejwt
```

### 4.4.2 Cấu hình

```
from rest_framework_simplejwt.views import TokenObtainPairView
```

### 4.4.3 Luồng

- User login → nhận token
- Gửi token trong header
- Các service verify token

## 4.5 Giao tiếp giữa các Service

### 4.5.1 REST API call

```
import requests

response = requests.get(
    "http://product-service:8001/products/"
)
```

### 4.5.2 Best Practice

- Timeout
- Retry
- Circuit breaker (advanced)

## 4.6 Docker hóa hệ thống

### 4.6.1 Dockerfile (Django)

```
FROM python:3.10
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

### 4.6.2 docker-compose.yml

```
version: '3'

services:
  user-service:
    build: ./user-service
    ports:
      - "8000:8000"

  product-service:
    build: ./product-service
    ports:
      - "8001:8001"
```

## 4.7 Luồng hệ thống (End-to-End)

### 4.7.1 Use case: Mua hàng

1. User login (user-service)
2. Xem sản phẩm (product-service)
3. Add to cart (cart-service)
4. Tạo order (order-service)
5. Thanh toán (payment-service)
6. Giao hàng (shipping-service)

### 4.7.2 Sequence logic

- order-service gọi payment-service
- payment success → gọi shipping-service

## 4.8 Triển khai Kubernetes (Optional)

### 4.8.1 Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: user-service
```

### 4.8.2 Service

```
kind: Service
spec:
  type: ClusterIP
```

## 4.9 Logging và Monitoring

- Logging: ELK stack
- Monitoring: Prometheus + Grafana

## 4.10 Đánh giá hệ thống

### 4.10.1 Hiệu năng

- Response time
- Throughput

### 4.10.2 Khả năng mở rộng

- Scale từng service
- Load balancing

### 4.10.3 Ưu điểm

- Linh hoạt
- Dễ mở rộng

### 4.10.4 Nhược điểm

- Phức tạp triển khai
- Debug khó

## 4.11 Bài tập thực hành

- Triển khai các service bằng Django
- Kết nối qua API
- Docker hóa hệ thống
- Test full flow mua hàng + kết quả tư vấn

## 4.12 Checklist đánh giá

- Có API Gateway
- Có JWT Auth
- Có Docker chạy được
- Có flow order → payment → shipping

# Kết luận

- Microservices phù hợp hệ thống lớn
- DDD giúp thiết kế rõ ràng
- AI nâng cao trải nghiệm người dùng